

ABSTRACT

This project focuses on the design, deployment, and management of a web server using **Amazon Web Services (AWS)**, one of the world's leading cloud computing platforms. The primary goal was to understand and implement the process of hosting a website on the cloud using an **EC2 (Elastic Compute Cloud)** instance running **Ubuntu Linux** and the **Apache HTTP web server**. The implementation was carried out directly through the Ubuntu terminal within the AWS environment, without using SSH, thereby ensuring hands-on exposure to instance-based configuration and management.

The project began with the creation and configuration of an EC2 instance, including setting up key security parameters through **security groups** to allow inbound HTTP and HTTPS traffic. Following this, **Apache Web Server** was installed and configured to serve web pages over the internet. This allowed the team to deploy and test web applications in a real-time cloud environment, validating the operational performance of the server. Additional steps included updating system packages, verifying Apache service status, and testing accessibility through the instance's public IP address.

The key objectives of this project were to:

1. Understand the fundamentals of **cloud infrastructure** and **web hosting on AWS**.
2. Learn the configuration and deployment of **Apache HTTP Server** in a cloud environment.
3. Demonstrate the use of **AWS Management Console** for monitoring and managing instances.
4. Evaluate the **scalability, reliability, and efficiency** of deploying web services in the cloud.

Tools and technologies used include **AWS EC2**, **Ubuntu**, **Apache HTTP Server**, and **AWS Security Groups** for network management. The project outcomes revealed that cloud-based deployment offers high availability, simplified resource management, and cost-effective scalability compared to traditional hosting methods. Overall, the project provided valuable insights into **Infrastructure as a Service (IaaS)** and laid a strong foundation for advanced cloud computing and DevOps practices.

INDEX

SI NO	CONTENT	PAGE NO
1	INTRODUCTION	4
2	REQUIREMENT SPECIFICATION	7
3	OBJECTIVES	9
4	SYSTEM DESIGN & USER INTERFACE	12
5	METHODOLOGY	15
6	PROJECT ACTIVITIES	18
7	CONCLUSION	21
8	REFERENCES	22

INTRODUCTION

Internship Project Overview

The internship project titled “Web Server Deployment Using AWS and Apache” was undertaken to gain practical experience in cloud infrastructure management and web hosting technologies. Cloud computing has revolutionized how organizations deploy and manage applications by providing on-demand computing resources over the internet. Among the leading cloud service providers, Amazon Web Services (AWS) offers highly scalable and reliable infrastructure that enables users to host, deploy, and manage applications efficiently.

This project focused on deploying a web server using AWS EC2 (Elastic Compute Cloud) and Apache HTTP Server on an Ubuntu Linux environment. By setting up a cloud-based server, the project aimed to simulate a real-world scenario where a company hosts its website on the cloud to ensure high availability, flexibility, and cost efficiency. The work included configuring the instance, installing the Apache web server, managing network security through AWS security groups, and testing server accessibility through a public IP.

Existing studies and industrial practices show that organizations are rapidly transitioning from traditional on-premise hosting to cloud-based hosting due to the latter’s advantages in scalability, data security, and performance optimization. This project aligns with that trend by offering a hands-on understanding of Infrastructure as a Service (IaaS) and the operational management of cloud-hosted web applications.

PROBLEM INTRODUCTION

Traditional web hosting requires physical infrastructure such as servers, storage, and networking devices, which are expensive to set up and maintain. Such environments often face challenges including limited scalability, high operational costs, and downtime during maintenance or traffic spikes.

The problem identified was the lack of a scalable and cost-efficient hosting solution that can handle dynamic workloads while minimizing manual management. The project provides a solution by

leveraging AWS EC2 instances to host a web application on the cloud, thereby eliminating the need for physical infrastructure. By using Apache HTTP Server on the EC2 instance, the system ensures consistent performance and easy configuration for hosting dynamic and static web content.

This approach enables reliable service delivery, automatic scaling capabilities, and global accessibility—all while maintaining low operational overhead.

MODULE INTRODUCTION

The project was structured into several functional modules, each addressing a specific part of the deployment process:

1. AWS EC2 Instance Setup Module

- * Creation of an EC2 instance using the AWS Management Console.
- * Selection of Ubuntu as the operating system.
- * Configuration of instance details such as key pairs, storage, and network settings.

2. Security Configuration Module

- * Setup of AWS Security Groups to define inbound and outbound traffic rules.
- * Enabling HTTP (port 80) and HTTPS (port 443) access for public connectivity.
- * Ensuring the system remains protected from unauthorized access.

3. Apache Web Server Installation Module

- * Installation of the Apache HTTP Server directly through the Ubuntu terminal.

- * Configuration of web directories and verification of the Apache service status.
- * Deployment of test web pages to confirm successful hosting.

4. Testing and Verification Module

- * Testing the web application using the instance's public IP address.
- * Verifying successful server response and page rendering.
- * Ensuring continuous availability and monitoring performance metrics.

5. Project Documentation and Report Module

- * Documenting all steps, configurations, and outcomes for academic and professional evaluation.
- * Summarizing key findings, challenges faced, and lessons learned during deployment.

This modular design ensured a systematic workflow — from infrastructure setup to deployment and testing — while providing a clear understanding of each stage involved in cloud-based web hosting.

REQUIREMENT SPECIFICATION

The requirement specification defines the essential hardware, software, and development tools necessary for the successful design, deployment, and testing of the web server on AWS. These requirements ensure that the system operates efficiently in a cloud environment while providing stable web hosting performance.

SOFTWARE REQUIREMENTS

- **Operating System:** Ubuntu (Server Edition) – an open-source Linux distribution used as the base platform for server setup.
- **Web Server:** Apache HTTP Server – an open-source web server for hosting web content and applications.
- **Platform:** Amazon Web Services (AWS) – provides Elastic Compute Cloud (EC2) instances for deploying the server.
- **Web Browser:** Google Chrome or Mozilla Firefox – used to access and test the hosted web application.
- **Package Manager:** APT (Advanced Package Tool) – used to install and update software packages on Ubuntu.

HARDWARE REQUIREMENTS

- **Processor:** Minimum 2.0 GHz Dual Core or higher
- **Memory (RAM):** Minimum 2 GB (Recommended 4 GB)
- **Storage:** At least 20 GB of free disk space
- **Network:** Stable internet connection and an active AWS account
- **Display:** Minimum 1024 × 768 resolution for the AWS Management Console

SOFTWARE SPECIFICATIONS

- **Instance Type:** t2.micro (Free-tier eligible)
- **Operating System Image (AMI):** Ubuntu Server 22.04 LTS
- **Web Server Ports:** HTTP (Port 80) and HTTPS (Port 443)
- **Security Configuration:** AWS Security Groups for inbound and outbound rule management
- **Public IP Address:** Used to access the hosted website from a browser
- **User Privileges:** Root access within the EC2 instance for installation and configuration

DEVELOPMENT TOOLS

AWS Management Console: Used for creating, configuring, and monitoring EC2 instances.

Ubuntu Terminal: Used to execute installation and configuration commands.

Apache Utilities: Tools and commands used for managing and testing hosted sites.

Text Editor (nano or vim): Used for editing configuration and HTML files.

Web Browser: Used to test and verify the hosted website's functionality.

SYSTEM REQUIREMENT SUMMARY

This project utilizes a cloud-based virtual environment instead of traditional local hardware. By combining AWS EC2, Ubuntu, and Apache, the system provides reliable and scalable web hosting capabilities. The configuration ensures easy deployment, cost efficiency, and practical understanding of cloud infrastructure and management.

OBJECTIVES

The primary objective of this internship project is to design and deploy a Gallery Website using Amazon Web Services (AWS) and Apache Web Server. The project focuses on understanding the complete process of creating a functional, cloud-hosted website that displays images in an organized gallery format.

The aim was not only to develop a visually appealing website but also to gain hands-on experience in cloud deployment, server configuration, web hosting, and security management using the AWS platform. Through this project, the intern enhanced practical skills in HTML, CSS, and basic scripting, along with Linux-based server management and cloud infrastructure operations.

The Existing System

In the existing setup, most personal or small-scale gallery websites are hosted on local systems or traditional shared hosting services. These environments often depend on fixed storage capacity and limited scalability. Updating or modifying the content requires manual intervention, and users must rely on local or third-party hosting providers to keep the site active.

Such systems do not fully utilize the potential of cloud technology, and performance may degrade under high traffic conditions.

Limitations of the Existing System

Limited Storage and Scalability: Local hosting restricts the ability to expand when more images or data are added.

Downtime and Maintenance Issues: Traditional servers need manual updates and frequent maintenance.

High Hosting Costs: Paid hosting services can become expensive for long-term use.

Limited Accessibility: Locally hosted websites are often not accessible globally.

Security Risks: Local or shared hosting lacks robust security measures and monitoring tools.

Proposed System

The proposed system introduces a cloud-based gallery website hosted on an AWS EC2 instance running Ubuntu Linux with the Apache HTTP Server.

The gallery website is developed using HTML, CSS, and basic scripting, featuring an organized layout of images, responsive design, and smooth navigation. The AWS platform provides a scalable and reliable infrastructure that ensures the website is accessible globally.

The system allows the user (administrator) to upload, update, and manage gallery images easily, while visitors can view them through a web browser using the EC2 public IP or domain. Security Groups in AWS manage network access to ensure safe and secure hosting.

Advantages of the Proposed System

Cloud-Based Hosting: Uses AWS EC2 for global accessibility and reduced hardware dependency.

Scalability: Easy to expand storage or resources when more images are added.

Low Maintenance: Minimal manual intervention with automated updates and monitoring.

Cost Efficiency: AWS free-tier or pay-as-you-go model minimizes hosting costs.

Enhanced Security: Controlled access through AWS Security Groups and cloud firewalls.

High Availability: Website remains online with minimal downtime.

User-Friendly Interface: Simple and clean design for easy image viewing and navigation.

Real-Time Access: The gallery can be accessed from any device connected to the internet.

This project demonstrates how a simple web gallery can be transformed into a professional, cloud-hosted solution using AWS and Apache. It builds both technical deployment skills and front-end web development capabilities, aligning with modern industry standards for cloud-based web applications.

SYSTEM DESIGN & USER INTERFACE

System Design Overview

The Gallery Website Project was designed to demonstrate the deployment of a cloud-based web application using Amazon Web Services (AWS) and Apache Web Server. The system consists of a front-end gallery interface hosted on an EC2 instance running Ubuntu Linux, with Apache HTTP Server used to serve web pages to users over the internet.

The project follows a client-server architecture, where users access the website through a browser, and the server (EC2 instance) responds with the requested web pages and image content. The gallery website was created using HTML and CSS, ensuring a clean, responsive, and user-friendly layout suitable for various devices.

System Architecture

The architecture of the system can be described in three main layers:

Presentation Layer (Front-End):

Designed using HTML and CSS.

Displays images in a grid or gallery format with smooth navigation.

Includes a home page, gallery page, and image details section.

Application Layer (Server-Side Configuration):

Managed through the Apache HTTP server hosted on AWS EC2.

Handles client requests (HTTP/HTTPS) and delivers web content.

Responsible for managing directory structure, access permissions, and content rendering.

Cloud Infrastructure Layer (AWS Environment):

EC2 instance acts as the virtual host.

Security groups manage inbound and outbound traffic.

Elastic IP ensures stable connectivity for public access.

Scope of the Project

The scope of the project focuses on end-to-end development, deployment, and testing of a functional gallery website hosted on a cloud platform. It covers all necessary steps from setting up the environment to presenting the final deliverable.

Specific tasks and activities performed during the internship included:

Creating and configuring an AWS EC2 instance using the Ubuntu operating system.

Installing and setting up the Apache HTTP Server for web hosting.

Designing and developing the gallery website front-end using HTML and CSS.

Uploading and organizing image files into web directories for display.

Configuring AWS Security Groups to enable HTTP (port 80) access.

Testing website accessibility using the public IP address of the EC2 instance.

Verifying Apache service status and ensuring successful content delivery.

Documenting all project activities, configurations, and outputs for the final report.

Deliverables

The key deliverables produced at the end of the internship project were:

Functional Gallery Website – A responsive website hosted on AWS, capable of displaying images in a structured format.

Configured AWS EC2 Instance – A fully operational cloud server running Ubuntu and Apache.

Project Documentation – Detailed report including project objectives, configurations, outputs, and conclusions.

Demonstration Output – Verification of the gallery website's accessibility through a public IP, confirming successful hosting.

User Interface Design

The user interface (UI) of the gallery website is simple, clean, and intuitive, ensuring a pleasant browsing experience for users.

Key features of the UI include:

A Gallery Page displaying multiple images in an organized grid layout.

Navigation Menu allowing users to switch between sections easily.

Responsive Design that adjusts automatically for mobile, tablet, and desktop screens

Minimal Aesthetic with focus on image content, using basic HTML/CSS styling and layout control.

Design Goals

The main design goals were:

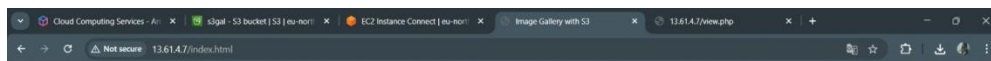
To create a lightweight and fast-loading image gallery.

To ensure accessibility and performance through cloud hosting.

To provide an attractive yet minimalistic user interface.

To gain hands-on knowledge of both web design and cloud deployment.

USER INTERFACE:

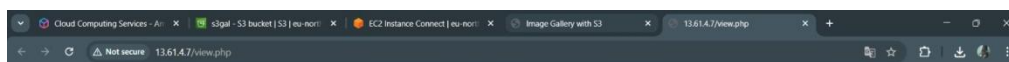


Upload Image

Choose File No file chosen Upload

Gallery

[View Image gallery](#)



Gallery



METHODOLOGY

The methodology outlines the systematic approach followed to design, develop, and deploy the Gallery Website using Amazon Web Services (AWS) and Apache Web Server. The project was executed in a step-by-step manner to ensure clarity, accuracy, and successful cloud deployment. The development process followed a linear sequential model (also known as the Waterfall Model) where each stage was completed before moving on to the next.

Project Approach

The overall approach of this project focused on combining web development and cloud deployment skills.

The methodology included the following major stages:

Planning and Requirement Analysis:

Identified the goal of creating a responsive gallery website.

Collected requirements such as software, hardware, and AWS tools.

Defined the scope of the project and planned resource usage within the AWS free-tier.

System Setup and Environment Configuration:

Created an AWS EC2 instance using Ubuntu (Linux) as the operating system.

Configured Security Groups to allow inbound HTTP traffic (port 80).

Accessed the instance through the AWS Management Console and Ubuntu terminal.

Installation and Configuration of Apache Web Server:

Installed Apache HTTP Server using the APT package manager.

Verified server status and enabled it to start automatically on boot.

Tested default Apache web page through the instance's public IP address.

Web Development Phase:

Designed the gallery website using HTML and CSS.

Created multiple web pages including Home, Gallery, and About sections.

Organized images into folders and linked them within the HTML structure.

Ensured responsive design using flexible CSS layouts and media queries.

Deployment Phase:

Uploaded the project files to the Apache server's root directory (/var/www/html).

Restarted the Apache service to reflect changes.

Verified successful hosting by accessing the website through the EC2 instance's public IP.

Testing and Validation:

Checked for website accessibility, layout consistency, and image loading performance.

Tested from multiple browsers and devices to ensure cross-platform compatibility.

Validated server uptime and Apache response using monitoring tools.

Documentation and Presentation:

Recorded all configuration commands, setup steps, and deployment results.

Prepared comprehensive documentation describing system design, objectives, and outcomes.

Tools and Technologies Used

Amazon Web Services (AWS): For cloud hosting and resource management.

AWS EC2 (Elastic Compute Cloud): Virtual machine to host the website.

Ubuntu Linux: Operating system environment for server deployment.

Apache HTTP Server: To serve and manage web content.

HTML and CSS: For front-end web design and layout styling.

Web Browser (Chrome/Firefox): To access and test the hosted website.

AWS Security Groups: To configure inbound and outbound network traffic.

Methodological Framework

The methodology followed a practical implementation-oriented framework, ensuring that theoretical cloud concepts were applied through real-world configuration. Each phase emphasized hands-on learning — from instance creation and server setup to deployment and verification.

This structured approach ensured that the Gallery Website was developed, hosted, and tested successfully on AWS, achieving the objectives of scalability, accessibility, and real-time performance.

PROJECT ACTIVITIES

During the internship, several key activities were undertaken to successfully complete the development and deployment of the Gallery Website using AWS. Each activity was focused on understanding real-world cloud infrastructure, website hosting, and application deployment through practical implementation. The following stages outline the complete workflow of the project:

1. Requirement Analysis

The project began with gathering and analyzing requirements to determine the software, hardware, and networking components needed for hosting a website on AWS. This phase involved:

Identifying the purpose and design layout of the gallery website.

Choosing AWS EC2 (Ubuntu) as the hosting platform.

Selecting Apache HTTP Server for web service deployment.

Ensuring all security, accessibility, and scalability requirements were met.

2. Instance Creation and Configuration

An EC2 instance was created on the AWS Management Console with the following key activities:

Selecting the Ubuntu operating system.

Configuring instance type (t2.micro – Free Tier eligible).

Setting up Security Groups to allow HTTP (port 80) and HTTPS (port 443) traffic.

Allocating and associating a public IP address for internet access.

Launching the instance and accessing it through the Ubuntu terminal within AWS.

3. Installation of Apache Web Server

Once the EC2 instance was running, the Apache HTTP server was installed and configured directly through the Ubuntu terminal using the following steps:

Updating system packages using `sudo apt update`.

Installing Apache using `sudo apt install apache2`.

Starting and enabling the Apache service to run automatically at startup.

Testing the installation by accessing the instance's public IP through a web browser.

4. Website Development

The Gallery Website was developed using basic front-end web technologies:

HTML and CSS for layout design and styling.

Image optimization techniques to ensure faster loading.

A simple, user-friendly interface for displaying the image gallery.

Responsive design to make the gallery accessible on both desktop and mobile devices.

5. Deployment on AWS

The developed gallery website was deployed on the EC2 instance by:

Uploading web files to the Apache server's root directory (/var/www/html).

Verifying that the site was correctly served using the instance's public IP address.

Testing accessibility, image display, and page responsiveness across browsers.

6. Testing and Validation

Extensive testing was performed to ensure the website functioned correctly and the server was stable:

Checking server uptime and Apache service status.

Testing website accessibility from different devices and networks.

Monitoring performance using AWS Management Console metrics.

7. Troubleshooting and Optimization

Several issues encountered during deployment, such as permission errors and network configuration problems, were resolved through:

Adjusting file permissions for the Apache directory.

Reconfiguring security groups to allow necessary inbound traffic.

Optimizing image sizes and HTML structure to improve website speed.

8. Documentation and Reporting

The final phase involved preparing project documentation detailing:

Step-by-step setup procedures.

System configurations and command-line operations.

Screenshots of the AWS environment, website interface, and testing results.

Summary of outcomes and learning experiences during the internship.

CONCLUSION

The internship project on “Gallery Website Deployment using Amazon Web Services (AWS)” provided a comprehensive understanding of cloud computing concepts, web hosting, and server management. Through this project, the main objectives of designing, developing, and deploying a functional gallery website on a cloud-based platform were successfully achieved.

The implementation process—starting from creating and configuring an EC2 instance, installing and managing the Apache web server, and deploying the website content—offered practical exposure to real-world cloud infrastructure management. Directly working within the Ubuntu terminal on AWS without relying on SSH helped build a deeper understanding of instance-level configurations and security management.

The project also enhanced technical skills in web design (HTML/CSS), Linux command-line operations, and AWS environment handling, while strengthening problem-solving abilities through troubleshooting and performance optimization activities. By deploying the gallery website online, the project successfully demonstrated the reliability, scalability, and efficiency of cloud-based web hosting.

Overall, this internship experience provided valuable hands-on knowledge of Infrastructure as a Service (IaaS), improved confidence in cloud deployment workflows, and laid a strong foundation for future learning in DevOps, system administration, and web application development.

REFERENCES

Amazon Web Services (AWS) Documentation

Purpose: Used for understanding EC2 instance setup, security groups, and Apache deployment on AWS.

Link: <https://docs.aws.amazon.com/ec2/>

Ubuntu Official Documentation

Purpose: Referred for Linux command-line operations and Apache installation procedures.

Link: <https://ubuntu.com/server/docs>

Apache HTTP Server Project Documentation

Purpose: Used to learn about configuration, virtual hosts, and web directory management.

Link: <https://httpd.apache.org/docs/>

W3Schools – HTML and CSS Tutorials

Purpose: For front-end design and responsive layout of the gallery web pages.

Link: <https://www.w3schools.com/>

Stack Overflow Community Discussions

Purpose: For resolving technical issues and troubleshooting AWS and Apache configuration errors.

Link: <https://stackoverflow.com/>

YouTube – AWS EC2 & Apache Setup Tutorials

Purpose: Visual learning and step-by-step demonstration of AWS web hosting.

Link: <https://www.youtube.com/>

MDN Web Docs (Mozilla Developer Network)

Purpose: For HTML5 and CSS3 code reference, structure, and optimization techniques.

Link: <https://developer.mozilla.org/>